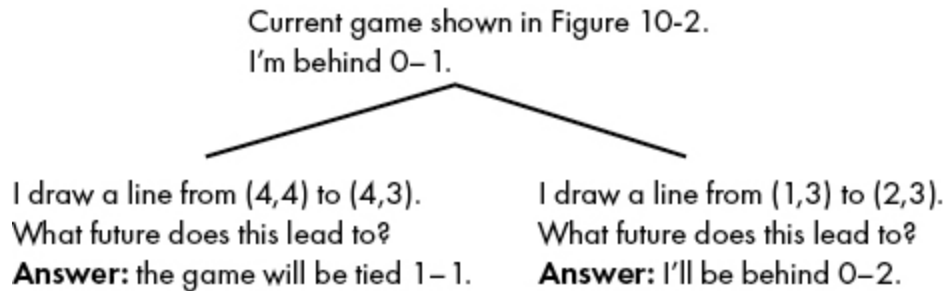


Now we know what future will be brought about by the two moves. [Figure 10-7](#) notes these futures in an updated version of [Figure 10-4](#).

Because we know exactly what future to expect from each of our two moves, we can do a maximization: the move that leads to the maximum, the best score, is the move on the left, so we choose that one.



[Figure 10-7](#): Using Figures 10-5 and 10-6, we can reason about the futures that each move will lead to and then compare them.

The reasoning process we just went through is known as the minimax algorithm. Our decision in the present is about maximizing our score. But in order to maximize our score, we have to consider all the ways that our opponent will try to minimize our score. So the best choice is a maximum of minima.

Note that minimax goes through time in reverse. The game proceeds forward in time, from the present to the future. But in a way, the minimax algorithm proceeds backward in time, because we consider the scores of possible far futures first and then work our way back to the present to find the current choice that will lead to the best future. In the context of our game tree, the minimax code starts at the top of the tree. It calls itself recursively on each of its child branches. The child branches, in turn, call minimax recursively on their own child branches. This recursive calling continues all the way to the terminal nodes, where, instead of calling minimax again, we calculate the game score for each node. So we're calculating the game score for the terminal nodes first; we're starting our game score calculations in the far future. These scores are then passed back to their parent nodes so that the parent nodes can calculate the best moves and corresponding score for their part of the game. These scores and moves are passed back up through the game tree until arriving back at the very top, the parent node, which represents the present.